Contents

Chapter 1

HTML

HTML, an abbreviation for Hypertext Markup Language, is the standard language for creating web pages and web applications. It is a cornerstone technology of the World Wide Web and forms the structure and layout of web content.

1.1 HTML Elements

An HTML document is composed of a series of elements, which are denoted by tags. Elements have an opening tag and a closing tag with content in between. Some elements, however, are self-closing and do not contain any content.

An element consists of three parts:

- **Opening tag**: The opening tag is the start of an element. The format of it is `<tagname>`.
- **Content**: Content is the data or information between the tags.
- **Closing tag**: The closing tag indicates the end of an element. The format of it is `</tagname>`.

For example, the paragraph tag `<p>` is used to denote a paragraph:

```
1 <p>This is a paragraph.</p>
```

This applies to a lot of HTML elements but not every one. There are HTML elements that are self-closing. A self-closing element does not require a closing tag. The image tag `<img>` is an example of a self-closing element.

```
1 <img src="image.jpg" alt="Description" />
```

1.2 Attributes

Attributes provide additional information about HTML elements. For some elements like `img`, they are required for the element to function correctly. They are always specified in the opening tag and consist of a name and value pair.

The syntax for attributes is:

```
<tagname attribute="value">Content</tagname>
```

There are so many attributes available, we cannot cover them all here. However, some common attributes that work on most elements include:

- **id**: The `id` attribute specifies a unique identifier for an element (should be unique within the entire page)
- **class**: The `class` attribute specifies one or more class names for styling with CSS
- **style**: The `style` attribute specifies inline CSS styling
- **title**: The `title` attribute specifies additional information displayed as a tooltip

Examples:

```
<p id="intro" class="highlight">
    Introduction paragraph
</p>

<div style="color: blue; font-size: 16px;">
    Styled text
</div>

<img src="photo.jpg" title="My Photo" alt="Me">
```

1.3 HTML Document Structure

A typical HTML document has a specific structure. Every HTML5 document begins with a DOCTYPE declaration, which tells the browser which version of HTML to expect.

The basic structure of an HTML document is:

```
<!DOCTYPE html>
<html>
<head>
    <title>Page Title</title>
</head>
<body>
    <!-- Content goes here -->
</body>
```

```
</html>
```

The main components are:

- **DOCTYPE declaration**: This informs the browser about the version of HTML. For HTML5, it is `<!DOCTYPE html>`.
- **html**: The `html` element is the root element that wraps all other content.
- **head**: This element contains metadata and information about the page, including:
  - `<title>`: This element sets the title of the page, which appears in the browser tab.
  - `<meta>`: This element includes metadata such as character encoding, and viewport settings.
  - `<link>`: This element is used to link external resources like CSS stylesheets.
  - `<script>`: This element is used to include JavaScript code or external JavaScript files.
- **body**: The `body` element contains all the visible content of the web page.

Here is a more complete example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Website</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Welcome</h1>
    <p>This is my website.</p>
</body>
</html>
```

1.4 Web Development and HTML

The Web Development trio consists of three essential languages that work together to create modern web applications. These languages are HTML, CSS, and JavaScript.

- **HTML**: HTML provides the structure and content of web pages. It defines what elements exist on the page. Think of it as the skeleton and meat of a web page.

- **CSS**: Cascading Style Sheets (CSS) handles the presentation and styling. It controls how elements look, including colors, fonts, spacing, and layout. Think of it as the clothing and makeup that makes the page visually appealing.
- **JavaScript**: JavaScript adds interactivity and behavior. It enables dynamic content changes, form validation, animations, and user interactions. Think of it as the brain that allows the page to respond to actions like clicking a button or submitting a form.

1.5 Metadata

Metadata is information about the HTML document that is not directly displayed on the page. The information is provided through a series of `<meta>` tags with certain attributes and their corresponding values. These are placed in the `<head>` section and provides important information to browsers, search engines, and developers.

Common metadata elements include:

- `<meta charset="UTF-8">`: This line specifies the character encoding of the document. UTF-8 is the standard encoding for web pages.
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">`: This line configures the viewport for responsive design. This tells mobile browsers how to scale the page.
- `<meta name="description" content="...">`: This line provides a brief description of the page content for search engines.
- `<meta name="keywords" content="...">`: This line lists relevant keywords for search engine optimization (SEO).
- `<meta name="author" content="...">`: This line specifies the author of the document.

Example of using multiple meta tags:

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="A tutorial about web development">
    <meta name="keywords" content="HTML, CSS, JavaScript">
    <meta name="author" content="John Doe">
    <title>Web Development Tutorial</title>
</head>
```

1.6 Div Elements

The `<div>` element is a generic container used to group content together. The name "div" stands for "division." `<div>` has no specific meaning to it. It is a neutral wrapper used for layout, styling, or organizing content with CSS and JavaScript.

The `<div>` element is one of the most commonly used HTML elements in existing applications. It is often used with `class` or `id` attributes to apply styling or add behavior.

Here's a basic example:

```
<div class="container">
    <h2>Section Title</h2>
    <p>This is some content inside a div.</p>
</div>
```

As mentioned, it is one of the most commonly used HTML elements. Here's an example with multiple divs for layout:

```
<div class="header">
    <h1>Website Title</h1>
</div>

<div class="content">
    <p>Main content goes here.</p>
</div>

<div class="footer">
    <p>&copy; 2024 My Website</p>
</div>
```

1.7 Semantic HTML

Unlike `<div>` elements, Semantic HTML elements are elements that clearly describe their meaning to both the browser and the developer. This improves code readability, helps with search engine optimization (SEO), and makes pages more accessible to assistive technologies like screen readers.

Semantic elements follow a logical structure that reflects the content of the page. Some common semantic elements include:

- `<header>`: The `header` element represents the top section of the page or a section, typically containing the logo, navigation, or introductory content.

- `<nav>`: The `nav` element contains navigation links to other pages or sections.
- `<main>`: The `main` element specifies the main content area of the page.
- `<article>`: The `article` element represents independent, self-contained content that could stand alone, such as a blog post or news article.
- `<section>`: The `section` element defines a generic section of content. This is preferred instead of using multiple divs for grouping related content.
- `<aside>`: The `aside` element contains supplementary content like sidebars, ads, or related links.
- `<footer>`: The `footer` element represents the bottom section of a page or section.

Here is an example of using semantic elements to structure a simple blog page:

```
<header>
    <h1>My Blog</h1>
    <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
    </nav>
</header>

<main>
    <article>
        <h2>Article Title</h2>
        <p>Article content here...</p>
    </article>

    <aside>
        <h3>Related Posts</h3>
        <ul>
            <li><a href="#post1">Post 1</a></li>
            <li><a href="#post2">Post 2</a></li>
        </ul>
    </aside>
</main>

<footer>
    <p>&copy; 2026 My Blog. All rights reserved.</p>
</footer>
```

1.8 Headings and Sections

HTML provides six levels of headings, ranging from `<h1>` to `<h6>`. Headings establish a hierarchical structure for page content. `<h1>` is the most important and typically used once per page for the main title, while `<h6>` is the least important.

```
<h1>Main Page Title</h1>
<h2>Major Section</h2>
<h3>Subsection</h3>
<h4>Sub-subsection</h4>
<h5>Minor heading</h5>
<h6>Least important heading</h6>
```

Headings are important for defining the structure of the content and improving accessibility. They allow users to quickly scan the page and understand the organization of information. Proper use of headings also benefits SEO by signaling the importance of content to search engines.

1.9 Text Elements

HTML provides several elements for formatting text content. Here is a list of some of the most commonly used ones:

- `<p>`: The `p` element defines a paragraph. Browsers automatically add space before and after.
- `<br>`: The `br` element inserts a line break. This is a self-closing element.
- `<hr>`: The `hr` element displays a horizontal line. This is a self-closing element.
- `<strong>`: The `strong` element indicates strong importance. This is typically displayed in bold.
- `<em>`: The `em` element indicates emphasis. This is typically displayed in italics.
- `<b>`: The `b` element displays bold text without semantic importance.
- `<i>`: The `i` element displays italic text without semantic emphasis.
- `<mark>`: The `mark` element highlights text with a yellow background.
- `<code>`: The `code` element displays text in monospace font for code.

Examples:

```
<p>This is a <strong>important</strong> point.</p>

<p>This word is <em>emphasized</em>.</p>

<p><code>var x = 5;</code></p>

<p>This is <mark>highlighted</mark> text.</p>
```

1.10 Links

Links are created using the `<a>` (anchor) element with the `href` attribute specifying the destination.

```
<a href="https://www.example.com">Visit Example</a>
```

Links can point to various destinations:

- **External websites**: `<a href="https://www.google.com">External Site</a>`. This will navigate the user to Google.
- **Other pages within the same site**: `<a href="about.html">About</a>`. This will navigate the user to the about.html page within the same website.
- **Email addresses**: `<a href="mailto:email@example.com">Email</a>`. This will open the user's email to send an email to the specified address.
- **Phone numbers**: `<a href="tel:+1234567890">Call Us</a>`. This will initiate a phone call on devices that support it. This can be useful for mobile users.
- **Specific locations on the same page**: `<a href="#section2">Jump to Section 2</a>` (where `<h2 id="section2">Section 2</h2>`). When the link is clicked, the page will scroll to the header with the id "section2".

Apart from the `href` attribute, here are some other useful attributes for links:

- `target="_blank"`: This attribute and value pair opens the link in a new tab.
- `target="_self"`: This attribute and value pair opens the link in the same tab. This is the normal behavior if the target attribute is not specified.
- `title="tooltip"`: This attribute and value pair displays text when the user hovers over the link like a tooltip.

1.11 Lists

HTML supports two main types of lists to organize content. These include unordered lists and ordered lists. Both types of lists use the `<li>` element to define individual list items.

1.11.1 Unordered Lists

Unordered lists display items with bullet points. As the name suggests, the order of items is not important like the items of a shopping list. They use the `<ul>` tag with list items defined by `<li>`:

```
<ul>
    <li>Apples</li>
    <li>Bananas</li>
    <li>Oranges</li>
</ul>
```

This would render an unordered list as:

- Apples
- Bananas
- Oranges

1.11.2 Ordered Lists

Ordered lists display items with numbers or other counters. As the name suggests, the order of items is important, like steps in a recipe. They use the `<ol>` tag:

```
<ol>
    <li>First step</li>
    <li>Second step</li>
    <li>Third step</li>
</ol>
```

The output of the above code would be:

1. First step
2. Second step

3. Third step

The `<ol>` tag supports the `type` attribute to customize or change the default numbering style. So instead of numbers, you can use letters or Roman numerals. Some of the common attributes include:

- `type="a"`: Lowercase letters (a, b, c...).
- `type="A"`: Uppercase letters (A, B, C...).
- `type="i"`: Lowercase Roman numerals (i, ii, iii...)
- `type="I"`: Uppercase Roman numerals (I, II, III...)

1.12 Navigation

The `<nav>` element is used to wrap navigational links within a navigation section. It's a semantic element that helps structure the page and signal to assistive technologies that this section contains navigation. This is commonly used for site menus, table of contents, or any set of links that help users navigate the website.

```
<nav>
    <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/about">About</a></li>
        <li><a href="/services">Services</a></li>
        <li><a href="/contact">Contact</a></li>
    </ul>
</nav>
```

1.13 Tables

Tables organize data into rows and columns. They are created using the `<table>` tag. However, that is used as a container for table elements necessary to render the layout of the table. Those table elements include:

- `<tr>`: The `tr` element defines a table row
- `<th>`: The `th` element defines a table header cell. By default, it is displayed in bold and centered. It is used to indicate that the cell is a header for a column or row.
- `<td>`: The `td` element defines a table data cell. It is used for regular cells that contain data.

- `<thead>`: The `thead` element groups header rows.
- `<tbody>`: The `tbody` element groups body rows.
- `<tfoot>`: The `tfoot` element groups footer rows.
- `<caption>`: The `caption` element provides a title for the table.

Here's a basic table example:

```
<table>
    <caption>Monthly Sales</caption>
    <thead>
        <tr>
            <th>Month</th>
            <th>Sales</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>January</td>
            <td>$5000</td>
        </tr>
        <tr>
            <td>February</td>
            <td>$6000</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <th>Total</th>
            <td>$11000</td>
        </tr>
    </tfoot>
</table>
```

The output of the above code would be:

Table 1.1: Monthly Sales

Month	**Sales**
January	$5000
February	$6000
Total	$11000

There are attributes that can be applied to table cells to make them span multiple rows or columns. These attributes are `rowspan` (row span) and `colspan` (column span). These attributes are used to merge cells together, which can be useful for creating more complex table layouts. `colspan` is used to make a cell span across multiple columns, while `rowspan`

is used to make a cell span across multiple rows. The value of these attributes specifies the number of rows or columns the cell should span.

Here's an example of how to use them:

```
<table>
    <tr>
        <th colspan="2">Personal Information</th>
    </tr>
    <tr>
        <td>Name</td>
        <td>John Doe</td>
    </tr>
    <tr>
        <th>Age</th>
        <th rowspan="2">Skills</th>
    </tr>
    <tr>
        <td>30</td>
    </tr>
    <tr>
        <td colspan="2">HTML, CSS, JavaScript</td>
    </tr>
</table>
```

The output of the above code would be:

Personal Information	
Name	John Doe
Age	Skills (spans 2 rows)
HTML, CSS, JavaScript	

1.14 Forms

Forms allow users to input and submit data. The `<form>` element contains form controls and is submitted to a server to process the provided data. The most common form controls include text fields, checkboxes, radio buttons, and submit buttons. These form controls are created using input elements.

Here's an example of a basic form structure with various input elements:

```
<form>
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>
```

```
4
5       <label for="email">Email:</label>
6       <input type="email" id="email" name="email">
7
8       <input type="submit" value="Submit">
9   </form>
```

The `label` element is used to provide a label for form controls. The `for` attribute of the label should match the `id` of the corresponding input element to associate them together.

Input fields have various types, each serving a different purpose. The "type" attribute indicates the type of input field. Here's a list of the most common input types:

- `type="text"`: This is a single-line text input.
- `type="email"`: This is a single-line email input with validation that ensures the entered value is a valid email address.
- `type="password"`: This is a single-line password input (text is hidden).
- `type="number"`: This is a single-line numeric input.
- `type="checkbox"`: This is a checkbox for multiple selections.
- `type="radio"`: This is a radio button for single selection.
- `type="file"`: This is a file upload input.
- `type="date"`: This is a date picker input.
- `type="submit"`: This is a submit button.
- `type="reset"`: This is a reset button that will clear all form fields.

There are more form control elements besides the "input" element. Other form elements include:

- `<textarea>`: This is a multi-line text input.
- `<select>`: This is a dropdown menu. A dropdown is a list of options that can be expanded to show all available options.
- `<option>`: These are options within a dropdown.
- `<fieldset>`: This groups related form elements.
- `<legend>`: This is the title for a fieldset.

Here's an example that includes various form controls and uses fieldsets to group related

information:

```
<form>
    <fieldset>
        <legend>Personal Information</legend>

        <label for="username">Username:</label>
        <input type="text" id="username" name="username">

        <label for="password">Password:</label>
        <input type="password" id="password" name="password">
    </fieldset>

    <fieldset>
        <legend>Preferences</legend>

        <label>
            <input type="checkbox" name="newsletter">
            Subscribe to newsletter
        </label>

        <label for="country">Country:</label>
        <select id="country" name="country">
            <option>Select a country</option>
            <option value="usa">USA</option>
            <option value="uk">UK</option>
            <option value="ca">Canada</option>
        </select>
    </fieldset>

    <button type="submit">Submit</button>
</form>
```

1.15 Images

Images are added using the `<img>` element, which is self-closing.

The basic syntax for the `<img>` element is:

```
<img src="image.jpg" alt="Description of image">
```

The most essential attributes for the `<img>` element are:

- `src`: The path to the image file. This is a required attribute that specifies the source of the image. The source path can route to an image file within the application or

an external URL similar to a link. If the image cannot be found at the specified path, the browser will display a broken image icon. The `src` attribute is crucial for the image to load properly on the webpage. It is important to ensure that the path is correct and that the image file exists at that location.

- `alt`: This is alternative text displayed if the image cannot load or for screen readers. The `alt` attribute should provide a concise description of the image content.

Images come in various sizes. To control the display size of an image, you can use the `width` and `height` attributes.

- `width`: This specifies the width of the image in pixels or percentage.
- `height`: This specifies the height of the image in pixels or percentage.

Here's an example of using the `width` and `height` attributes to specify the dimensions of an image:

```
<img src="logo.png" alt="Company Logo" width="200" height="100">
```

CHAPTER 2

Introduction to Text

In computer systems, text is represented in files as a sequence of characters, each of which corresponds to a specific number, known as a character code. These character codes are then stored in the file as binary data.

2.1 Newlines and Carriage Returns

Two of the character codes that have special meanings are the newline (often represented as '\n') and the carriage return (often represented as '\r').

The newline character signifies the end of a line of text and the beginning of a new one. If you parse a text file with a Python script, it will see your 'enter' key press as a new line symbol. The carriage return character moves the cursor to the beginning of the line. The use of these characters can vary between operating systems. Unix-based systems (like Linux and MacOS) use the newline character to indicate the end of a line, while Windows systems use a combination of a carriage return and a newline ('\r\n').

2.2 ASCII

The American Standard Code for Information Interchange (ASCII) is one of the earliest character encodings. It uses 7 bits to represent each character, allowing it to define up to $2^7 = 128$ different characters. These include the English alphabet (in both lower and upper cases), digits, punctuation symbols, control characters (like newline and carriage return), and some other symbols.

2.3 UTF-8

UTF-8 (8-bit Unicode Transformation Format) is a variable-width character encoding that can represent every character in the Unicode standard, yet remains backward-compatible with ASCII. For the ASCII range (0-127), UTF-8 is identical to ASCII. However, it can use additional bytes (up to 4 bytes in total) to represent characters that are not included in ASCII, such as characters from other languages, emojis, and many other symbols. This has made UTF-8 a widely used encoding in many modern systems.

Chapter 3

Stop Words

Chapter 4

Stemming and Lemmatization

Stemming and lemmatization are two fundamental techniques in natural language processing that are used to prepare text data. They help in reducing inflectional forms of a word to a common base form.

4.0.1 Stemming

Stemming is the process of reducing a word to its word stem (its basic form). For example, a stemming algorithm would reduce the words ”jumping”, ”jumped”, and ”jumps” to the stem ”jump”.

It’s important to note that stemming may not always lead to actual words. For example, the stem of the word ”running” could be ”run” depending on the stemming algorithm used.

Stemming is generally simpler and faster than lemmatization, but it is also less precise.

4.0.2 Lemmatization

Lemmatization, on the other hand, reduces words to their base or root form, which is linguistically correct. For example, ”running” and ”runs” are both changed to ”run”.

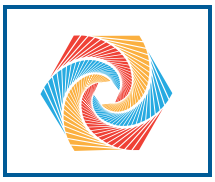
Lemmatization uses a more complex approach to achieve this. It considers the morphological analysis of the words and requires detailed dictionaries, which the algorithm can look through to link the form back to its lemma.

4.0.3 Applications

To summarize, both stemming and lemmatization help in text normalization and preprocessing, but while stemming can be faster and simpler, lemmatization is more accurate, as it uses more informed analysis to create groups of words with similar meanings based on the context. These are both used to improve natural language processing (NLP) in software such as search engines, machine learning models, algorithmic searches, and even baseline programs for autofill and text prediction.

Appendix A

Answers to Exercises



Index